AES Benchmarking on Raspberry Pi 5

Group H: Zach Agle, Abigail Jeffie, Juliana Neemeh, Jacob Dominguez

Cal Poly Pomona

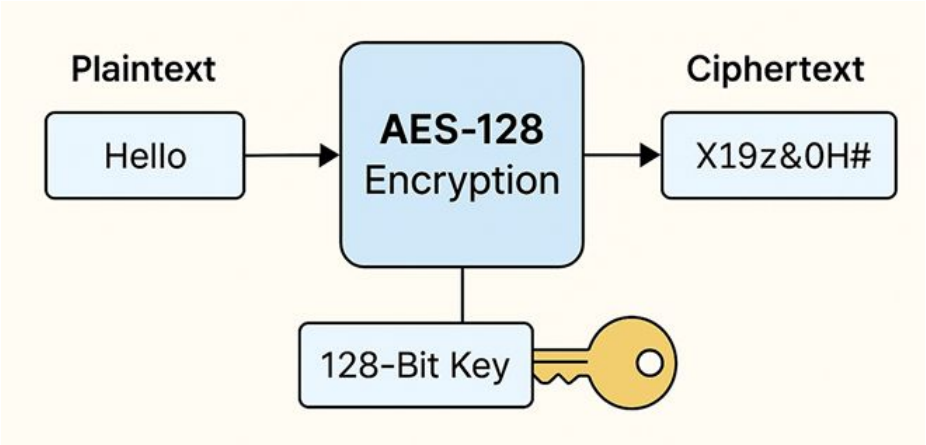
Project Details

<u>Goal:</u> Analyze how architectural implementation choices affect encryption performance on the Raspberry Pi 5.

We will benchmark hardware-accelerated AES on the Raspberry Pi 5 using two independent measurement methods: OpenSSL and cryptsetup both paired with the perf stat.

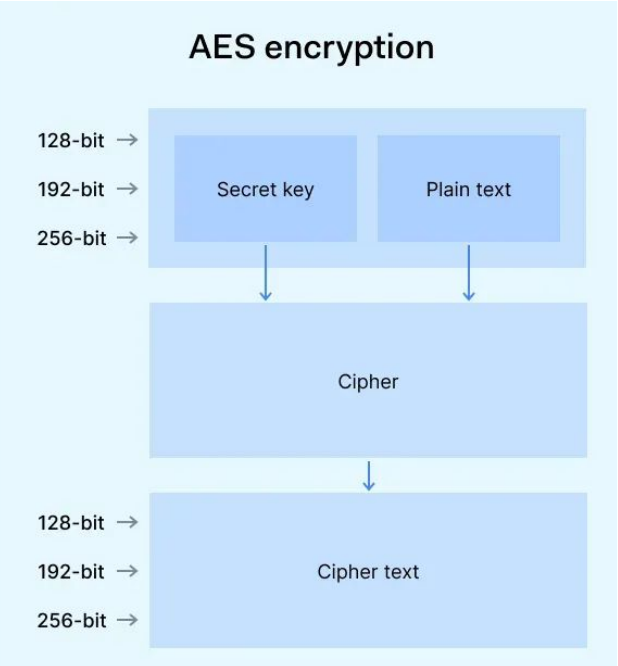
<u>Measured Values:</u>

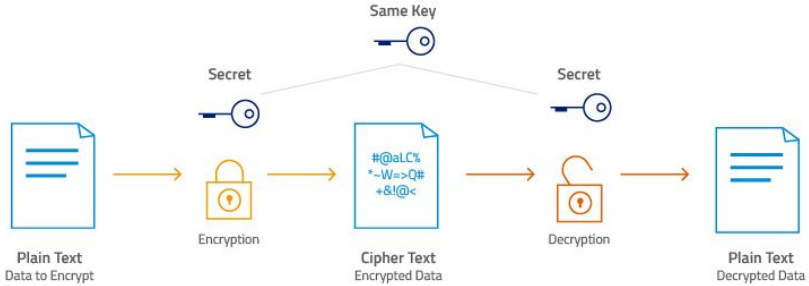
- Throughput (MB/s)
- CPU cycles
- Instructions executed
- Instructions per Cycle
- Cache miss rate (%)
- Branch miss rate (%)



Advanced Encryption Standard

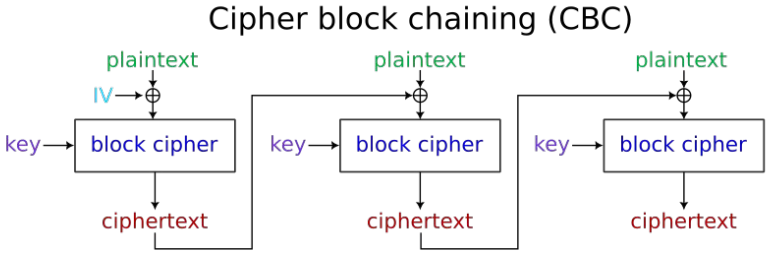
- A symmetric-key encryption algorithm used to secure digital information.
- Encrypts data by converting readable plaintext into unreadable ciphertext, and decrypts it back using the same secret key.
- Transforms plaintext into ciphertext through repeated rounds of substitution, shifting, mixing, and key-based transformations
- Uses keys lengths of 128, 192, or 256 bits (longer keys → stronger security)
- Used in applications like online banking, messaging apps, and protecting sensitive files
- Example:
  - Plaintext: HELLO → Ciphertext: XJQZP → Decrypted bacl to HELLO

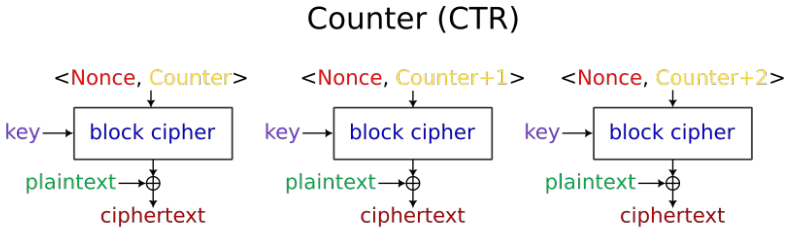




AES Modes Being Benchmarked:

- **CBC (Cipher Block Chaining):** encrypts each data block using the previous block's output.
- **CTR (Counter Mode):** encrypts each data block independently using a counter and a unique value.
- **GCM (Galois/Counter Mode):** extends CTR with authentication, verifying data integrity alongside encryption in a single operation.



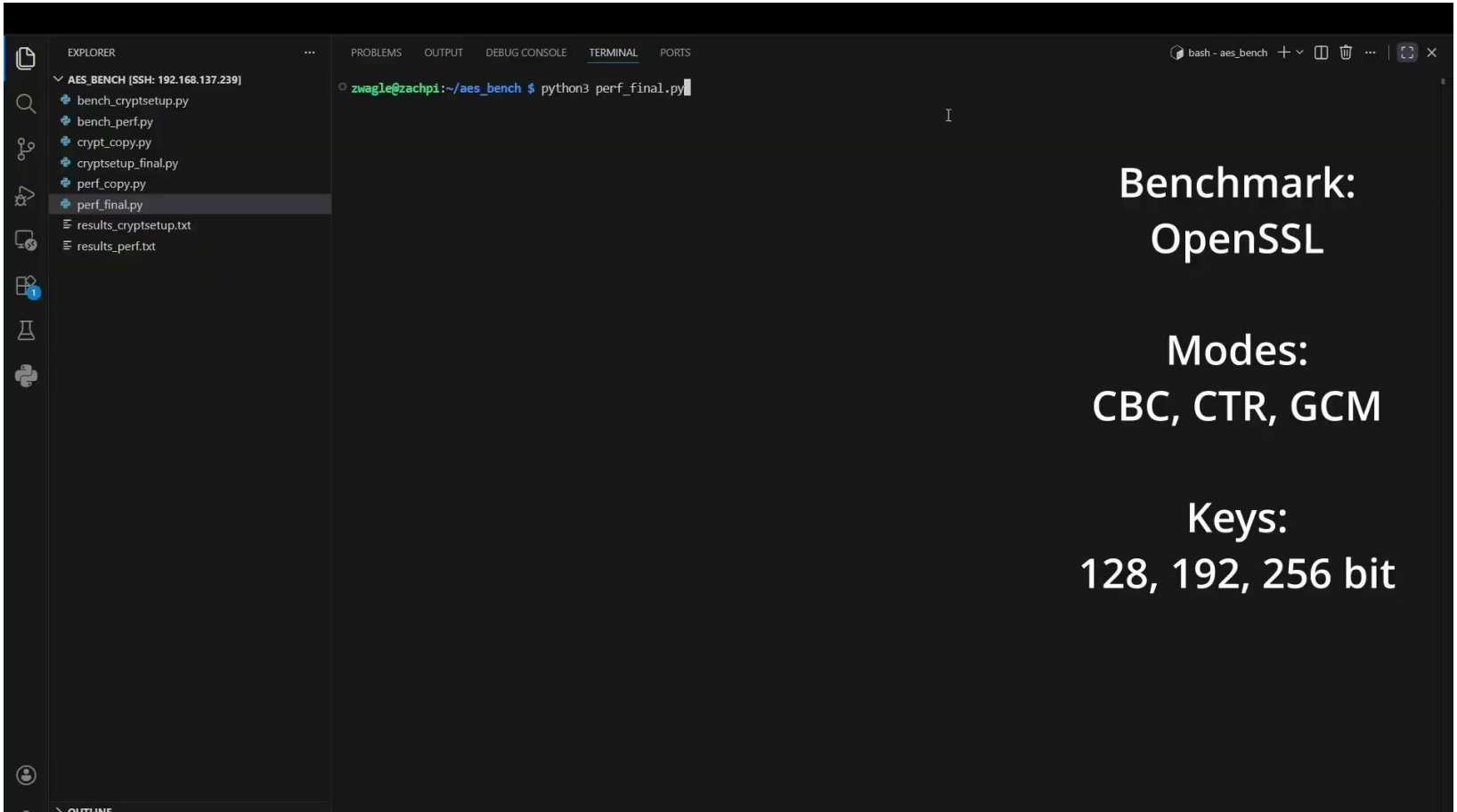


Benchmark Tools

- Benchmarks AES encryption performance using different libraries, modes, and key sizes.
  - **Cryptsetup**
    - Modes: CBC, CTR
    - Key sizes: 128, 192, and 256-bit
  - **OpenSSL**
    - Modes: CBC, CTR, GCM
    - Key sizes: 128, 192, and 256-bit
- Collect CPU performance data using stat perf:
  - Data collected: cycles, instructions, cache references and misses, branch references and misses
- Converts CPU metrics into per-second rates.
- Monitors system temperature and checks permissions.

LUKS
Linux Unified Key Setup

Source Code Demo



Project Blockers

perf stat permission restrictions:

- Linux by default restricts perf to userspace only.
- Cryptsetup AES executes almost entirely in kernel mode and the values read near zero.
- Setting **perf_event_paranoid=-1** removes the restriction, allowing perf to count events across both userspace and kernel mode.

cryptsetup does not support GCM:

- cryptsetup benchmark targets disk encryption, while GCM is used in communication protocols, making it outside cryptsetup scope.
- OpenSSL supports GCM through its own userspace library.
- Thus, GCM encryption could not be compared with both benchmark tools.

Terminal Command Fix:

sudo sh -c 'echo -1 > /proc/sys/kernel/perf_event_paranoid'

Comparing Data

cryptsetup benchmark

CBC Mode (Encrypt / Decrypt)
128-bit: 1141 / 2264 MiB/s
192-bit: 1054 / 2042 MiB/s
256-bit: 947 / 1832 MiB/s

CTR Mode (Encrypt / Decrypt)
128-bit: 2170 / 2179 MiB/s
192-bit: 1896 / 1897 MiB/s
256-bit: 1717 / 1712 MiB/s

Instructions per Cycle (IPC)
CBC: 1.52-1.55 | CTR: 2.10-2.18

Cache Miss Rate
CBC: ~2.6% | CTR: ~2.9%

perf stat + OpenSSL

CBC Throughput (Enc / Dec, 16KB blocks)
128-bit: 1911 / 3494 MiB/s
192-bit: 1595 / 2976 MiB/s
256-bit: 1367 / 2559 MiB/s

CTR Throughput (Enc / Dec, 16KB blocks)
128-bit: 3226 / 3224 MiB/s
192-bit: 2781 / 2782 MiB/s
256-bit: 2414 / 2414 MiB/s

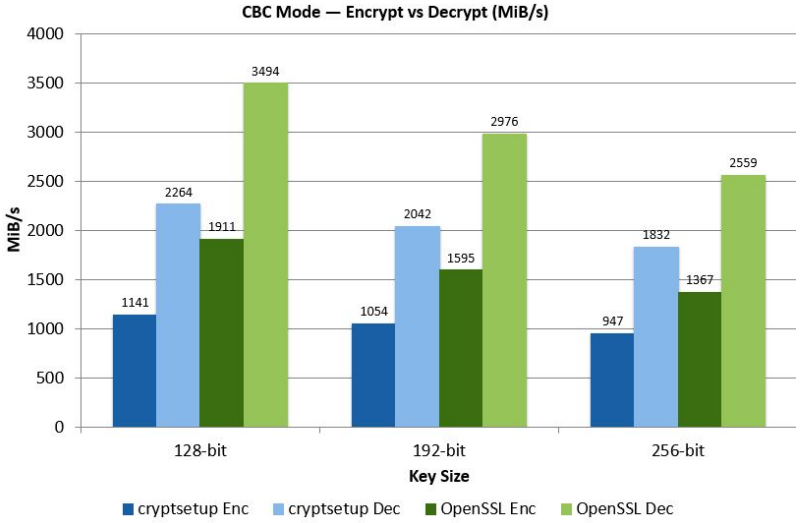
GCM Throughput (Enc / Dec, 16KB blocks)
128-bit: 2243 / 2208 MiB/s
192-bit: 1995 / 2000 MiB/s
256-bit: 1817 / 1842 MiB/s

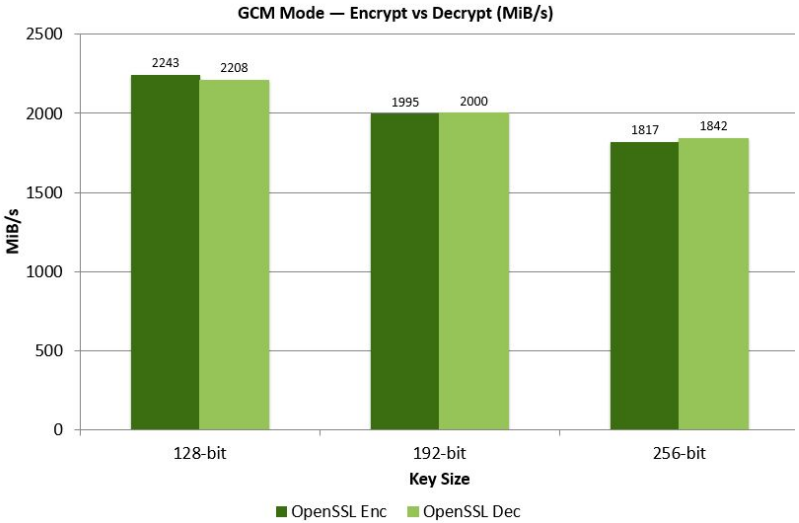
Instructions per Cycle (IPC)
CBC: 2.02–2.92 | CTR: 2.98–3.06 | GCM: 2.80–2.87

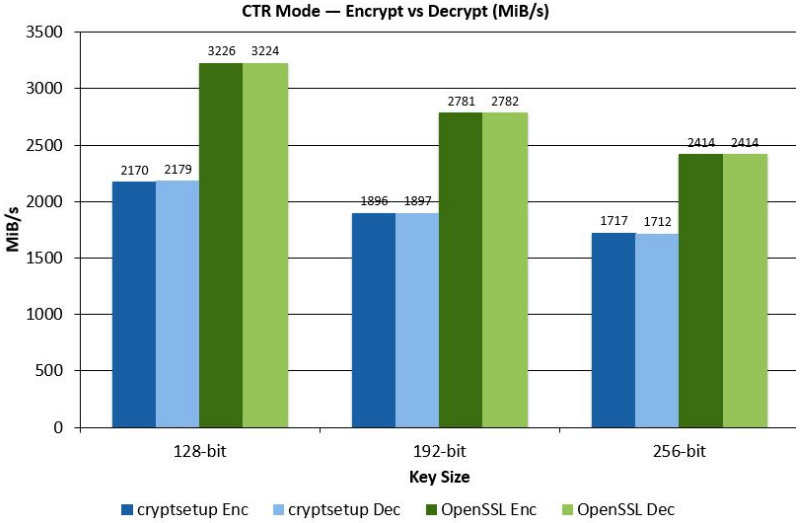
Cache Miss Rate
CBC/CTR: ≈0.00% | GCM dec: 0.01–0.21%

Key Observations

- CTR mode is overall the fastest of the modes tested.
- GCM sits between CBC and CTR despite adding authentication.
- CBC decrypt is nearly 2× faster than encrypt because blocks can be decrypted in parallel, encrypt cannot be parallelized

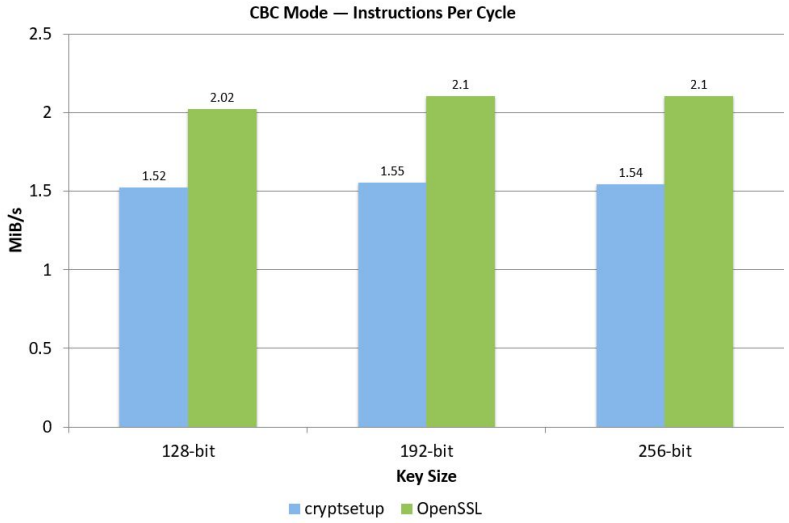


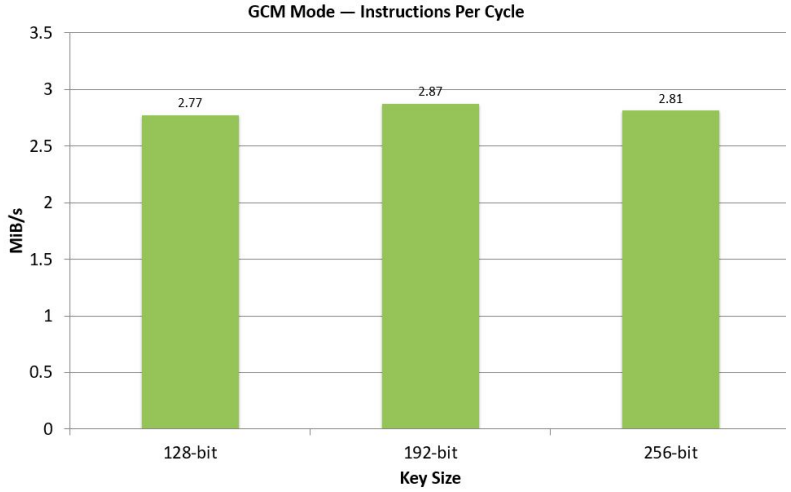


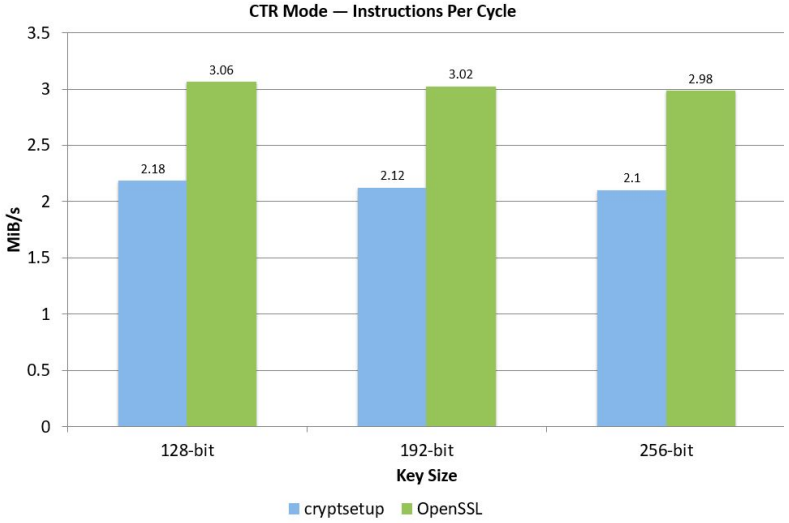


- For CTR and GCM modes, a keystream is generated during encryption. To decrypt, this keystream is XOR with the ciphered data, resulting in equal encrypt and decrypt speeds.

- OpenSSL is faster because it runs as a program in user space and has the ability to tune buffer sizes and memory layout purely for speed. Cryptsetup is constrained to the kernel's fixed disk sizes and prioritizes compatibility over throughput.







- Because cryptsetup is bound by the kernel's fixed sizes and strict memory, the CPU spends more cycles waiting on data, whereas OpenSSL's large buffers and optimization achieves nearly twice as many instructions out of every clock cycle.

Thank You

Any Questions?

AES Benchmarking on Raspberry Pi 5

Group H: Zach Agle, Abigail Jeffie, Juliana Neemeh, Jacob Dominguez

Project Details

Our project's goal was to analyze how architectural implementation choices affect encryption performance on the Raspberry Pi 5. We benchmarked hardware-accelerated AES on the Raspberry Pi 5 using perf paired with OpenSSL and cryptsetup. We used these programs in order to measure throughput, CPU cycles, instructions executed, instructions per cycle, cache miss rate, and temperature of the system.

Background

AES is a symmetric-key encryption algorithm used to secure digital information through encrypting readable plaintext into unreadable ciphertext, and decrypting it back using the same secret key. This transformation is done through repeated rounds of substitution, shifting, mixing, and key-based transformations. Different key lengths of of 128, 192, or 256 bits are used with longer key lengths ensuring stronger security. It is used in applications such as online banking, messaging apps, and protecting sensitive files.

Results

cryptsetup benchmark

CBC Mode (Encrypt / Decrypt)
128-bit: 1141 / 2264 MiB/s
192-bit: 1054 / 2042 MiB/s
256-bit: 947 / 1832 MiB/s

CTR Mode (Encrypt / Decrypt)
128-bit: 2170 / 2179 MiB/s
192-bit: 1896 / 1897 MiB/s
256-bit: 1717 / 1712 MiB/s

CPI (Cycles/Instruction)
CBC: 0.65–0.66 | CTR: 0.46–0.48

Cache Miss Rate
CBC: ~2.6% | CTR: ~2.9%

perf stat + OpenSSL

CBC Mode (CPI)
128: 0.50 | 192: 0.48 | 256: 0.48

CTR Mode (CPI)
128: 0.33 | 192: 0.33 | 256: 0.34

GCM Mode (CPI) — auth overhead
128: 0.36 | 192: 0.35 | 256: 0.36

Cache Miss Rate (all modes)
≈0.00% — near-perfect L1/L2 hit rate

Branch Miss Rate
CBC/CTR: ≈0.01% | GCM: 0.00–0.28%

Importance

AES has a significant impact in preventing sensitive information from being stolen or hacked. This project furthered our knowledge in AES and cybersecurity.

Takeaways

We found that the CTR cipher mode is the fastest, CTR is the fastest mode, usually above 2 GiB/s using the 128-bit key size. On the other hand, the CBC decrypt is ~2x encrypt due to hardware parallelism. Another result we observed is that using OpenSSL and perf shows near-zero cache misses where on the other hand, cryptsetup has around 2.6–2.9% since longer OpenSSL test windows warm caches fully. Lastly, we found that the GCM CPI, usually between 0.35–0.36 falls between CBC and CTR since GHASH authentication adds instructions but stays highly efficient